

“계산 로봇” 문제 풀이

작성자: 박현민

부분문제 1

문제의 정의상 제일 왼쪽 열에 있는 로봇의 입력 값은 0 하나인데, 열의 개수가 1이므로 모든 로봇의 저장 값도 0이다. 따라서 답은 항상 0이다.

부분문제 2

부분문제 1에 이어, 제일 왼쪽 열에 있는 로봇의 출력 값은 모든 $i(1 \leq i \leq M)$ 에 대해 $D_{i,1}$ 임을 알 수 있다. 그렇다면 2번째 열의 저장 값은 어떻게 정해지는지 살펴보자.

- 좌표 $(1, 2)$ 에 있는 로봇의 저장 값: $D_{1,1}, D_{2,1}$ 중 최댓값
- 좌표 $(2, 2)$ 에 있는 로봇의 저장 값: $D_{1,1}, D_{2,1}, D_{3,1}$ 중 최댓값
- 좌표 $(3, 2)$ 에 있는 로봇의 저장 값: $D_{2,1}, D_{3,1}, D_{4,1}$ 중 최댓값
- 좌표 $(4, 2)$ 에 있는 로봇의 저장 값: $D_{3,1}, D_{4,1}, D_{5,1}$ 중 최댓값
- ...
- 좌표 $(M-1, 2)$ 에 있는 로봇의 저장 값: $D_{M-2,1}, D_{M-1,1}, D_{M,1}$ 중 최댓값
- 좌표 $(M, 2)$ 에 있는 로봇의 저장 값: $D_{M-1,1}, D_{M,1}$ 중 최댓값

부분문제 1에서 제일 왼쪽 열에 있는 모든 로봇의 저장 값은 0임을 보았다. 따라서 이 부분문제의 답은 $i(1 \leq i \leq M)$ 에 대해 좌표 $(i, 2)$ 에 있는 로봇들의 저장 값 중 최댓값과 같고, 이는 위에서 살펴본 것에 따라 $D_{i,1}$ 중 최댓값과도 같다.

부분문제 3

왼쪽부터 각 로봇의 저장 값과 출력 값이 어떻게 정해지는지 살펴보자.

- 좌표 $(1, 1)$ 에 있는 로봇: 저장 값은 문제의 정의에 의해 0이고, 출력 값은 $D_{1,1}$ 이다.
- 좌표 $(1, 2)$ 에 있는 로봇: 저장 값은 $D_{1,1}$, 출력 값은 $D_{1,1} + D_{1,2}$ 이다.
- 좌표 $(1, 3)$ 에 있는 로봇: 저장 값은 $D_{1,1}, (D_{1,1} + D_{1,2})$ 중 최댓값인데, 모든 가중치는 양수이므로 $D_{1,1} + D_{1,2}$ 이다. 따라서 출력 값은 $D_{1,1} + D_{1,2} + D_{1,3}$ 이다.
- 좌표 $(1, 4)$ 에 있는 로봇: 저장 값은 $D_{1,1}, (D_{1,1} + D_{1,2}), (D_{1,1} + D_{1,2} + D_{1,3})$ 중 최댓값인데, 모든 가중치는 양수이므로 가장 마지막 값이다. 출력 값은 역시 이 값에 $D_{1,4}$ 를 더한 값이다.
- ...

마지막으로 좌표 $(1, M)$ 에 있는 로봇의 저장 값은 $D_{1,1}, (D_{1,1} + D_{1,2}), (D_{1,1} + D_{1,2} + D_{1,3}), \dots, (D_{1,1} + D_{1,2} + \dots + D_{1,M-2}), (D_{1,1} + D_{1,2} + \dots + D_{1,M-1})$ 중 최댓값이고 이번에도 역시 가장 마지막 값이다. 답인 로봇들의 저장 값 중 최댓값도 이 값이다.

부분문제 4

문제에서 주어진 순서대로 $j(1 \leq j \leq N)$ 가 증가하는 방향으로 살펴본다. 각 좌표에 있는 로봇의 입력 값들이 모두 왼쪽에 주어졌고 이를 순회하며 최댓값을 찾아 저장 값으로 가져오는 과정은 $O(MN)$ 에 할 수 있다. 로봇이 총 MN 개 있으므로 전체 시간복잡도는 $O(M^2N^2)$ 이고, 이 부분문제의 제약조건 상 충분히 시간 내에 동작한다.

부분문제 5

각 로봇의 입력 값들을 관찰하면 영역이 중복된다는 점에서 다음과 같은 성질을 찾을 수 있다.

$1 \leq i \leq M, 1 < j \leq N$ 를 만족하는 모든 i, j 에 대해 좌표 (i, j) 에 있는 로봇의 저장 값은 다음 값들 중 최댓값이다:

- ($i > 1$ 인 경우) 좌표 $(i - 1, j)$ 에 있는 로봇의 출력 값
- 좌표 (i, j) 에 있는 로봇의 출력 값
- ($i < M$ 인 경우) 좌표 $(i + 1, j)$ 에 있는 로봇의 출력 값

위와 같이 계산하면 시간복잡도 $O(MN)$ 만에 문제를 해결할 수 있다.

“누적 거리” 문제 풀이

작성자: 이온조

부분문제 1

후보 모임 장소의 위치 q_j 가 주어질 때마다 누적 거리 $f(q_j)$ 를 반복문을 이용하여 계산하면 $O(NQ)$ 의 시간복잡도로 부분문제 1을 해결할 수 있다.

부분문제 2

후보 모임 장소의 위치 q_j 에 대해서, KOI 나라에 존재하는 마을들을 두 집합 S, T 로 나누자. 집합 S 에는 q_j 보다 작거나 같은 좌표를 가진 마을들의 번호가 들어 있고, 집합 T 에는 q_j 보다 큰 좌표를 가진 마을들의 번호가 들어 있다. 이 때 누적 거리 $f(q_j) = \sum_{i=1}^N a_i |x_i - q_j|$ 에 대한 식을 다음과 같이 절댓값 기호 없이 풀어 쓸 수 있다.

$$f(q_j) = \sum_{i=1}^N a_i |x_i - q_j| = \sum_{s \in S} a_s (q_j - x_s) + \sum_{t \in T} a_t (x_t - q_j) = \left(\sum_{s \in S} a_s - \sum_{t \in T} a_t \right) q_j - \sum_{s \in S} a_s x_s + \sum_{t \in T} a_t x_t$$

좌표의 범위가 1부터 200 000까지이기 때문에, a_i 에 대한 부분합 배열과 $a_i x_i$ 에 대한 부분합 배열을 미리 계산할 수 있다. $\sum_{s \in S} a_s$ 과 $\sum_{t \in T} a_t$ 는 a_i 에 대한 부분합 배열을 사용해 $O(1)$ 시간에 계산할 수 있고, $\sum_{s \in S} a_s x_s$ 와 $\sum_{t \in T} a_t x_t$ 역시 $a_i x_i$ 에 대한 부분합 배열을 사용해 $O(1)$ 시간에 계산할 수 있다. 그러므로 $f(q_j)$ 역시 $O(1)$ 시간에 해결 가능하고, $O((\text{좌표 범위}) + N + Q)$ 시간에 부분문제 2를 해결할 수 있다.

부분문제 3, 4

이 문제를 해결할 때 마을들의 좌표 x_i 를 정렬하여도 답에 영향을 끼치지 않으므로 마을들의 좌표를 모두 정렬하고 시작하였다고 가정하자.

부분문제 2와 마찬가지로 a_i 와 $a_i x_i$ 에 대한 부분합 배열을 구축하되, 배열의 인덱스에는 좌표값 대신 순서가 들어가게 하자. 집합 S, T 를 구분하는 마을의 번호를 이분 탐색 등의 알고리즘으로 $O(\log N)$ 시간에 구할 수 있고, 마찬가지로 방법으로 $f(q_j)$ 를 계산할 수 있다. 총 시간복잡도는 $O((N + Q) \log N)$ 이다.

이 과정에서 $a_i = 1$ 인 경우만 고려하여 x_i 에 대한 부분합 배열만 구축하고 이분 탐색 등을 이용하여 답을 계산할 경우 부분문제 3만 해결할 수 있으며, 모든 경우를 고려하여 풀이한 경우 부분문제 4까지 모두 해결할 수 있다.

“일이 이어져야 좋다” 문제 풀이

작성자: 이재웅

부분문제 1

부분문제 1은 N 이 2의 거듭제곱 꼴인 경우이다. 이 경우, S_N 은 1과 0이 한 개씩 교대로 나타내는 형태를 띄게 된다.

예를 들어, $S_1 = 1, S_2 = 101, S_4 = 1010101$ 과 같은 꼴이 되는 것을 확인할 수 있다.

따라서 질의 (i, j, k) 의 답은 i 나 j 가 홀수일 경우 $\min(2k+1, j-i+1)$ 이고, i, j 가 짝수일 경우 $\min(2k, j-i+1)$ 를 만족하게 된다.

이와 같이 특수한 형태의 N 에 대한 문자열의 성질을 관찰하면 $O(Q)$ 시간에 문제를 해결할 수 있다.

부분문제 2

부분문제 2는 N 이 1000 이하로 매우 작은 경우이다. 이때, 문자열 S_N 의 길이는 $2N - 1$ 이하임을 관찰함으로써 문자열의 길이 역시 매우 작음을 알아낼 수 있다.

분할 정복 등의 알고리즘을 통해 문자열의 길이에 대한 선형 시간에 문자열의 각 원소를 모두 알아낼 수 있고, 문자열의 길이가 $2N - 1$ 이하이므로 $O(N)$ 시간에 S_N 의 모든 원소를 구할 수 있다.

이제 문자열 S_N 의 원소를 알아낸 것을 바탕으로 어떻게 질의를 해결할 수 있을지 생각해보자.

우선, 시작점이 s 로 고정되어 있고 0을 최대 k 개까지 포함하는 $S_N[i..j]$ 의 가장 긴 부분문자열에 주목하자. 문자열의 모든 원소를 이미 알고 있는 상태라면, $S_N[s..e]$ 에 포함되는 0의 개수가 k 개를 초과하거나 e 가 j 보다 커지기 직전까지 부분문자열의 끝점 e 를 1씩 증가시켜 감으로써 조건을 만족하는 부분문자열을 구할 수 있다.

또한, 시작점이 $i, i+1, \dots, j$ 일 경우에 대해 조건을 만족하는 가장 긴 부분문자열의 길이를 구한 뒤, 그것들의 최댓값을 계산함으로써 질의 (i, j, k) 를 해결할 수 있다.

이때, 시작점이 s 일 경우 0을 최대 k 개까지 포함하는 가장 긴 부분문자열의 끝점의 위치가 s 가 커짐에 따라 단조 증가한다는 성질을 바탕으로, 투 포인터 기법을 통해 각각의 질의를 $O(j-i+1)$ 만에 해결할 수 있다.

이 경우 S_N 의 모든 원소를 계산하는 데 $O(N)$ 이 필요하고, Q 개의 질의를 각각 $O(N)$ 만에 해결하므로 총 $O(QN)$ 시간에 문제를 해결할 수 있다.

부분문제 3

부분문제 3은 $j-i+1$ 의 합이 100,000 이하인 경우이다. 따라서 $j-i+1$ 의 합이 N 에 비해 작다는 점을 이용해 $S_N[i..j]$ 의 원소를 알아내는 방법에 대해 생각해보자.

S_N 의 x 번째 원소 $S_N[x]$ 에 대해, $S_{\lfloor N/2 \rfloor}$ 의 길이가 L 이고 S_N 의 길이가 $2L+1$ 이라면 아래와 같은 식이 성립한다.

$$S_N[x] = \begin{cases} S_{\lfloor N/2 \rfloor}[x], & \text{if } x < L+1 \\ N\%2, & \text{if } x = L+1 \\ S_{\lfloor N/2 \rfloor}[x-L-1], & \text{if } x > L+1 \end{cases}$$

위 식을 이용하면 $S_{\lfloor N/2 \rfloor}[x]$ 를 $O(\log N)$ 에 계산할 수 있다.

$S_N[i..j]$ 의 모든 원소를 알고 있다면, 부분문제 2에서와 같이 해당 질의를 $O(j-i+1)$ 만에 해결할 수 있고, 주어진 문자열의 모든 원소를 구하는 데 $O((j-i+1) \log N)$ 이 필요하므로, 총 $O(\sum_{q=1}^Q (j_q - i_q + 1) \log N)$ 시간에 문제를 해결할 수 있다.

부분문제 4

부분문제 4는 모든 질의에 대해 $k = 0$ 인 경우이다.

먼저, 질의 (s, e, k) 에 대해, 가장 왼쪽부터 시작해서 연속한 1의 최대 길이, 가장 오른쪽부터 시작해서 연속한 1의 최대 길이, 구간 (s, e) 전체에서 연속한 1의 최대 길이를 각각 $L(s, e), R(s, e), A(s, e)$ 로 정의하자.

$$L(s, e) = \begin{cases} L(s, m-1), & \text{if } L(s, m-1) < m-s \\ (m-s) + L(m, e), & \text{if } L(s, m-1) = m-s \end{cases}$$

$$R(s, e) = \begin{cases} R(m, e), & \text{if } R(m, e) < e-m+1 \\ (e-m+1) + R(s, m-1), & \text{if } R(m, e) = e-m+1 \end{cases}$$

$$A(s, e) = \max(A(s, m-1), A(m, e), R(s, m-1) + L(m, e))$$

정의에 따라 위와 같은 식이 성립하므로, 우리는 각 질의에 대한 답 $A(s, e)$ 를 분할정복을 통해 찾을 수 있다.

단순히 위 식만 이용해 계산할 경우 $T(n) = 2T(n/2) + O(1)$ 로부터 각 질의당 $O(N)$ 의 시간이 걸림을 알 수 있지만, 이때 $S_N[s..e]$ 는 수많은 $S_{\lfloor N/2^i \rfloor}$ 꼴 문자열의 합으로 표현할 수 있고, 이를 바탕으로 모든 $S_{\lfloor N/2^i \rfloor}$ 에 대해 각각의 L, R, A 값을 미리 계산해 둬므로써 분할정복에 걸리는 시간을 $O(\log N)$ 으로 단축시킬 수 있다.

따라서 전체 질의를 총 $O(Q \log N)$ 시간에 해결할 수 있다.

부분문제 5

부분문제 5는 부분문제 4와 달리 k 가 0보다 큰 값을 가질 수 있으므로, L, R, A 의 정의에도 변화가 필요하다.

구간 (s, e) 의 가장 왼쪽부터 시작해서 0을 최대 k 개 포함하는 가장 긴 문자열의 길이를 $L(s, e, k)$

구간 (s, e) 의 가장 오른쪽부터 시작해서 0을 최대 k 개 포함하는 가장 긴 문자열의 길이를 $R(s, e, k)$

구간 (s, e) 의 부분 문자열 중 0을 최대 k 개 포함하는 가장 긴 문자열의 길이를 $A(s, e, k)$ 로 정의하자.

$$L(s, e, k) = \begin{cases} L(s, m-1, k), & \text{if } L(s, m-1, k) < m-s \\ (m-s) + L(m, e, k-k'), & \text{if } L(s, m-1, k') = m-s, \end{cases}$$

(k' 은 $L(s, m-1, x) = m-s$ 를 만족하는 가장 작은 x)

$$R(s, e) = \begin{cases} R(m, e, k), & \text{if } R(m, e, k) < e-m+1 \\ (e-m+1) + R(s, m-1, k-k'), & \text{if } R(m, e, k') = e-m+1 \end{cases}$$

(k' 은 $R(m, e, x) = e-m+1$ 을 만족하는 가장 작은 x)

$$A(s, e) = \max(A(s, m-1), A(m, e), \max_{k'=0}^k (R(s, m-1, k') + L(m, e, k-k')))$$

정의에 따라 위와 같은 식이 성립하므로, 우리는 각 질의에 대한 답 $A(s, e, k)$ 를 분할정복을 통해 찾을 수 있다.

부분문제 4와 같이 모든 $S_{\lfloor N/2^i \rfloor}$ 에 대해 각각의 L, R, A 값을 미리 계산해 둬으로써 분할정복에 걸리는 시간을 $O(\log N)$ 으로 단축시킬 있고, 이러한 전처리에는 각각의 질의마다 $O(k \log N)$ 의 시간이 필요하므로 총 $O((\sum_{q=1}^Q k + Q) \log N)$ 시간에 문제를 해결할 수 있다.

“공통 괄호 문자열 사전” 문제 풀이

작성자: 최은수

부분문제 1

S 의 부분 문자열인 괄호열을 S 의 부분 괄호열이라 하자. 모든 괄호열은 여는 괄호 하나에서 시작해 닫는 괄호 하나에서 끝나기 때문에 A 의 부분 괄호열의 개수는 많아야 $\frac{|A|^2}{4}$ 개이다. 마찬가지로, B 의 부분 괄호열의 개수는 많아야 $\frac{|B|^2}{4}$ 개이다.

A 의 부분 괄호열 중, B 의 부분 괄호열인 것만을 모아, 이를 정렬하여 K 번째의 문자열을 출력하면 $O(|A|^5 + |B|^5)$ 에 문제를 해결할 수 있다.

부분문제 2

A 의 부분 괄호열과 B 의 부분 괄호열 쌍 모두를 비교하는 것이 아니라, 각각을 정렬하여 비교하면 $O(|A|^3 + |B|^3)$ 에 문제를 해결할 수 있다.

부분문제 3

A 의 부분 괄호열 S, T 가 있어, S 가 T 의 접두사가 된다면, T 는 S 보다 사전순으로 크므로 답이 아니다. 따라서 A 의 각 접미사에 대해, 괄호열인 접두사 중 가장 짧은 문자열만 구하면 된다.

각 문자부터 시작하여, 괄호 문자열 판별을 적용하면 되므로 $O(|A|^2)$ 의 시간복잡도를 가진다.

부분문제 4

A 의 모든 접미사에 대한 트라이를 만들고, 이 중 올바른 괄호열인 문자열에 해당하는 노드에 표시를 하자.

B 의 모든 접미사를 트라이에서 검색하며, 검색 중 참조하는 모든 노드에 표시를 하자.

구성된 트라이를 dfs로 순회하자. 순회 중, 여는 괄호에 해당하는 자식을 닫는 괄호에 해당하는 자식보다 먼저 방문하면 이는 사전순이 된다. 따라서 순회 중 K 번째로 만나는 두 번 표시된 노드에 해당하는 문자열이 답이 된다.

트라이의 구성 및 검색에는 문자열의 길이의 합만큼의 시간이 소요되므로, 시간복잡도는 $O(|A|^2 + |B|^2)$ 이다.

부분문제 5, 6, 7, 8

두 문자열 A 와 B 를 합친 $A * B$ 의 접미사 배열을 구하자. 그 후, 이 접미사 배열의 LCP 배열을 구하자.

A 의 접미사 $A[i..|A|]$ 에 대해, 이 문자열의 B 의 부분 문자열인 가장 긴 접두사는 다음의 두 가지 경우이다.

- 사전순으로 $A[i..|A|]$ 보다 앞인 B 의 접미사 중 가장 뒤에 있는 접미사와의 최장 공통 접두사
- 사전순으로 $A[i..|A|]$ 보다 뒤인 B 의 접미사 중 가장 앞에 있는 접미사와의 최장 공통 접두사

두 경우의 길이 모두 스윙핑을 통하여 모든 접미사에 대해 $O(|A| + |B|)$ 에 구할 수 있으며, 둘 중 최댓값의 배열을 $l_1, l_2, \dots, l_{|A|}$ 라 하자.

또한, 어떤 문자열 S 가 있을 때, S 의 i 번째 문자까지 있는 여는 괄호의 개수에서 닫는 괄호의 개수를 뺀 값을 문자열 S 의 i 번째 높이 h_i 로 정의하자. 편의상 $h_0 = 0$ 으로 정의하자.

$A[i..j]$ 가 올바른 괄호열인 조건과, $h_{i-1} = h_j$ 이며, $i \leq k < j$ 를 만족하는 모든 k 에 대해 $h_k \geq h_j$ 인 조건은 동치이다.

$1 \leq i \leq |A|$ 를 만족하는 i 에 대해, 다음 방법을 통해 $A[i..j]$ 가 올바른 괄호열인 가장 큰 j 를 구할 수 있다.

먼저, 이분탐색을 통해 $i \leq k$ 이면서, $h_{i-1} - 1 = h_k$ 인 가장 작은 k 를 찾는다. 없다면 $k = |A|$ 로 둔다. $h_{i-1} = h_j$ 이며 $i \leq j \leq k$ 인 것은 $A[i..j]$ 가 올바른 괄호열인 것과 동치이다.

$A[i..j]$ 가 B 의 부분 문자열인 것은, $i \leq j < i + l_i$ 를 만족하는 것과 동치이다. 따라서 i 번째 문자에서 시작하는 A 의 부분 문자열 중 문제의 조건을 만족하는 것의 개수는, $i \leq j \leq \min(k, i + l_i - 1)$ 이며 $h_{i-1} = h_j$ 를 만족하는 j 의 개수이다.

A 의 접미사 배열의 순서대로 조건을 만족하는 부분 문자열의 개수를 세면서, K 개가 넘어갈 때 답을 구해 주면 된다.

주의해야 할 점은, 답은 중복을 허용하지 않는 집합에서의 K 번째 원소라는 점이다. 이 문제를 해결하는 방법은, 접미사 $A[i..|A|]$ 에 대한 계산을 할 때 이미 계산된 부분을 빼 주는 것이다. LCP 배열을 lcp_i 라 한다면, $i \leq j < i + lcp_i$ 에 해당하는 부분을 빼 주면 된다.

부분문제 5, 6, 7의 경우, 문제를 푸는 과정에서 일부의 처리를 건너뛰어도 해결할 수 있다.

- $A = B$ 인 경우, $l_i = |A| - i + 1$ 로 두어 해결할 수 있다.
- $K = 1$ 인 경우, 조건을 만족하는 j 중 가장 작은 하나만 보아 해결할 수 있다.

시간복잡도는 $O(|A| \log |A| + |B| \log |B|)$ 이다.